Universidad Abierta Interamericana
Facultad de Tecnología Informática
Ingeniería en Sistemas Informáticos

TRABAJO FINAL DE INGENIERÍA

Verificación del sistema

Revisión OWASP Top 10, cobertura de pruebas, medición del requisito de carga, auditoría de uso, revisión de coherencia y corrida de un clon limpio

Alumno: Pettini, Patricio Ezequiel
Legajo: B00072710-T1
Sede: Sede Norte — Boulogne
Docentes: Romero, Martín Miguel · Weingand, Gastón Matías

Versión 1.0 · 23/10/2026

Índice

Revisión contra OWASP Top 10 7
Resumen 7
A01 · Control de acceso roto 7
A02 · Fallas criptográficas 8
A03 · Inyección 9
A04 · Diseño inseguro 10
A05 · Configuración insegura — **cubierto** 10
A06 · Componentes vulnerables 12
A07 · Fallas de identificación y autenticación — parcial 13
A08 · Fallas de integridad de datos y software 13
A09 · Fallas de registro y monitoreo 14
A10 · Falsificación de peticiones del lado del servidor (SSRF) 14
Lo que esta revisión no es 15
Fuentes 15
Cobertura de pruebas 16
Medición 16
Por proyecto 16
Para qué sirve este número 16
Medición del requisito de carga 18
Cómo se midió 18
Resultado 18
Lo que apareció al limpiar 19
Concurrencia 19
Qué NO mide esto 20
Estabilidad bajo carga sostenida 21
Cómo se midió 21
Resultado 22
Lo que no mide 23

Auditoría de uso del sistema 24
Cómo se hizo 24
Veredicto 24
Lo que está bien, con la evidencia 25
La trazabilidad de la decisión automática 25
El aislamiento entre organizaciones 25
Los permisos 25
El contraste 26
La bitácora 26
Los reportes 26
El despliegue 26
Defectos encontrados y corregidos 27
1 · El solicitante no podía reportar nada · bloqueante 27
2 · Lo primero que veía el solicitante era un error de permiso 27
3 · La consola se llenaba de un error de React en cada pantalla 27
Observaciones abiertas 27
A · El sistema alerta sobre el 87 % del parque · la más importante 27
B · No se puede usar en un teléfono 28
C · Quien reporta no puede hacer nada más 28
D · El solicitante ve la consola del técnico 28
E · Las explicaciones están después de lo que explican 28
F · El texto es chico y la densidad no lo cambia 28
G · Detalles menores 29
La prueba del chico de 15 años 29
Valor para el cliente 30
Qué haría primero 30
Revisión de coherencia entre los documentos y el sistema 31
Qué se buscó 31
Resumen 31

1 · La sección financiera se contradecía a sí misma 31
2 · El desvío entre la meta y la proyección no estaba declarado 32
3 · El multiidioma no cumplía lo que el requisito no funcional promete 32
4 · Los manuales decían cuántas pruebas esperar, y no era ese número 33
5 · Nueve referencias cruzadas apuntaban a apartados que no existen 33
6 · El CP-15 no figuraba como caso de prueba 34
7 · El apartado 7.7.1 estaba antes del cuerpo de 7.7 34
8 · Seis títulos vacíos 34
9 · Basura de codificación en el título de 6.6.2 34
10 · El plan de sprints y el backlog no coincidían 34
11 · Una prueba enumeraba siete secciones de Configuración y hay nueve 35
12 · Una prueba del frontend estaba en rojo en el repositorio 35
13 · Los marcadores del texto de reserva no se reemplazaban 35
14 · El guion de la defensa daba respuestas que ya no son ciertas 35
15 · El panel de obra mostraba el proyecto de hace una semana 36
16 · El armado del paquete no se detenía con la suite en rojo 36
17 · La insignia de pruebas y el número de casos no se contaban 36
18 · Los recuentos del README 36
Lo que se verificó y estaba bien 37
Los tres controles nuevos 37
Corrida de prueba: un clon limpio del repositorio 39
Cómo se hizo 39
Resultado 39
Los tres hallazgos 40
1 · El repositorio no clonaba en Windows 40
2 · Siete pruebas fallaban en un clon limpio 40
3 · Dos copias del proyecto no conviven 41
Lo que hay que traer a mano 41
Lo que esta corrida no prueba 41

Fuentes..... 43

Revisión contra OWASP Top 10

> *PredictIT · Trabajo Final de Ingeniería · Patricio Pettini · B00072710-T1*

Esta revisión existe por una razón concreta: el ADR 0014 decide **no usar un gestor de identidad externo**. Sin un producto de terceros que se ocupe de la seguridad, la revisión no la hace nadie más.

Se recorren las diez categorías de OWASP (OWASP Foundation, 2021). Cada una dice qué hace el sistema, con el archivo donde está, y **qué le falta**. La columna que importa es la última: una revisión que sólo lista lo que está bien no es una revisión.

Resumen

	Categoría	Estado
A01	Control de acceso roto	Cubierto
A02	Fallas criptográficas	Cubierto · TLS activo en el despliegue, con certificado de Let's Encrypt
A03	Inyección	Cubierto
A04	Diseño inseguro	Cubierto
A05	Configuración insegura	Cubierto
A06	Componentes vulnerables	Cubierto · avisa, no corta
A07	Fallas de identificación	Parcial · **falta segundo factor**
A08	Fallas de integridad	Cubierto en datos y en las dependencias · **sin firma de artefactos**
A09	Fallas de registro	Cubierto
A10	SSRF	Cubierto

Queda abierta **A07**: falta el segundo factor. No está en el backlog como historia pendiente y es a propósito: es una consecuencia declarada de la decisión del **ADR 0014** —gestión de identidad propia sin IAM externo—, y la ruta de migración a un proveedor externo, que es donde el segundo factor llega sin construirlo, está escrita ahí.

A01 · Control de acceso roto

Qué hace el sistema. El control se aplica en **dos capas y en el orden correcto**. `[Authorize]` en el controlador exige token; la patente la exige la capa de negocio con `Exigir(...)`, que lanza `PermisoDenegadoException`. De los 81 endpoints, **3 son anónimos** y los tres están justificados:

el login, la lista de idiomas y el diccionario —la pantalla de inicio de sesión está traducida y ahí todavía no hay token—.

Los tres últimos en sumarse son los del alta de usuarios: crear y modificar exigen `USUARIO_GESTIONAR` —dar de alta es también dar roles—, y el cambio de contraseña propia no pide patente sino sesión, porque quien entra obligado a cambiarla todavía no tiene ninguna. Ese último comparte el límite por origen del login: verificar la contraseña actual es otra puerta para probarlas.

El aislamiento entre organizaciones se aplica en el acceso a datos, no en el controlador, a partir del contexto de sesión. La organización **nunca viaja en la petición HTTP**: si viajara, alcanzaría con cambiarla para leer los datos de otro cliente. Está en `SqlHelper` y en cada DAO, y lo verifica `AislamientoOrganizacionTests`.

La búsqueda de un usuario para operar sobre él pasa por la organización y no por el identificador (`UsuarioBusiness.BuscarDeLaOrganizacion`), que es el caso clásico de referencia directa insegura a objetos.

Qué falta. Nada identificado. Es la categoría mejor cubierta, y no por casualidad: el modelo de familias y patentes es el contenido central del trabajo.

A02 · Fallas criptográficas

Qué hace el sistema.

- **Contraseñas:** PBKDF2-HMAC-SHA256, **600.000 iteraciones** —la recomendación de la *OWASP Password Storage Cheat Sheet*—, sal de 16 bytes por usuario, en el formato `PBKDF2$600000$sal$hash`. Ver `HashContrasena`.

El número de iteraciones viaja adentro del hash, así que subirlo no invalida lo ya guardado: cada hash se verifica con las iteraciones con las que se derivó. Para que el cambio alcance a los usuarios viejos, el sistema **vuelve a derivar el hash cuando la persona inicia sesión**, que es el único momento en que la contraseña en claro existe. El proyecto venía con 120.000, que era el valor del sistema de referencia de 3.º año y quedaba por debajo de la recomendación; los datos iniciales ya se generan con 600.000.

Contra las dos referencias: el NIST pide **al menos 10.000** iteraciones para PBKDF2 (SP 800-63B, §5.1.1.2), de modo que el sistema está sesenta veces por encima de ese piso, y la hoja de recomendaciones de OWASP sugiere **600.000** para PBKDF2-HMAC-SHA256, que es exactamente el valor en uso. Cumple las dos.

Los ADR 0010 y 0014 describen 120.000 en su apartado de contexto, y ahí está bien: es el valor que el sistema tenía en la fecha de cada uno. Un ADR aprobado no se edita —lo fija el ADR 0001— y subir una constante no alcanza el criterio de significatividad que pide uno nuevo.

- **Credenciales de terceros:** AES-256-GCM con nonce nuevo en cada operación (`CifradoService`). La clave de API del proveedor de IA **entra y no vuelve a salir**: ningún endpoint la devuelve, ni truncada. Lo único consultable es su origen —BASE, ENTORNO o NINGUNA—.
- **La clave de firma del token** no está en el repositorio y el sistema **se niega a arrancar sin ella**, con al menos 32 bytes (`Program.cs`, líneas 36-49). No hay valor por defecto: un secreto con valor por defecto es un secreto público.

Sobre TLS. El cifrado en tránsito se termina en el proxy de adelante y no en la aplicación, que es lo habitual en un despliegue con contenedores. La configuración **está escrita y probada**: `frontend/nginx-tls.conf` trae el servidor de 443 con TLS 1.2 y 1.3, la lista de cifrados, la redirección desde el puerto 80 y HSTS a un año con los subdominios incluidos. Se activa montando el certificado y esa configuración, y el propio archivo trae el comando.

Y desde el 16/09/2026 está activa. El entorno de demostración corre en Microsoft Azure sobre `predictit-uai.mexicocentral.cloudapp.azure.com` con un certificado de Let's Encrypt, montando ese mismo archivo. La configuración se estrenó ahí: estaba escrita desde hacía semanas y decía de sí misma que existía «para que poner el sistema detrás de TLS sea montar dos archivos y cambiar una línea, y no un trabajo de investigación el día del despliegue». Fue exactamente eso, con una sola corrección: el primer certificado se pide **sin** TLS, así que quien responde el desafío de validación es `nginx.conf.template`, que no tenía la ubicación `/.well-known/acme-challenge/` y dejaba que la aplicación se comiera la petición.

En la ejecución local sigue apagada a propósito, y el motivo no cambió: en `localhost` el único certificado posible es autofirmado, y eso agrega una advertencia del navegador sin proteger nada. HSTS vive dentro del bloque de 443 y no en las cabeceras comunes porque mandarlo por HTTP no tiene efecto y confunde a quien lee la configuración.

Lo que queda pendiente de verdad es que un despliegue real obligue a que el certificado exista: hoy nada en el código lo verifica.

A03 · Inyección

Qué hace el sistema. Todo el SQL va por `SqlParameter`. No hay una sola concatenación de valores en una consulta, y no es por disciplina: el ADR 0008 habilita **`CA2100` como error de**

compilación, que es exactamente la regla que detecta SQL construido por concatenación. Con `TreatWarningsAsErrors`, un SQL armado a mano **no compila**.

El motor predictivo guarda sus parámetros como JSON en `ReglaAlerta.condicion` y los deserializa a tipos; no los interpola en ninguna consulta.

En el frontend, React escapa el contenido por defecto y el proyecto no usa `dangerouslySetInnerHTML` en ningún lugar.

Qué falta. Nada identificado.

A04 · Diseño inseguro

Qué hace el sistema.

- La **máquina de estados de la incidencia** vive en la capa de negocio (ADR 0012) y no en el formulario. Un cliente que arme el pedido a mano no puede saltar de Pendiente a Cerrada: la transición se valida del lado del servidor.
- Restaurar un respaldo pide que quien opera **escriba el nombre de la base**. Es la confirmación de que sabe qué va a reemplazar.
- El **disyuntor** ante la caída del proveedor de IA no falla abierto ni cierra el sistema: asigna por carga, marca la incidencia para revisión y lo asienta. Un sistema degradado que no lo dice deja al administrador sin motivo para revisar nada.
- El mensaje de error del login **no distingue** usuario inexistente de contraseña incorrecta. Distinguirlos le confirmaría a un atacante qué usuarios existen.

Qué falta. Nada identificado en el alcance actual.

A05 · Configuración insegura — **cubierto**

Qué hace el sistema. La clave de firma del token es obligatoria y sin valor por defecto. `appsettings.Development.json` y `appsettings.Local.json` están excluidos del repositorio. Swagger está apagado salvo en desarrollo o cuando se lo habilita explícitamente, y esa excepción está documentada en el código con su motivo.

La revisión anterior dejaba tres cosas abiertas. Las tres se cerraron:

1 · Ninguna contraseña tiene valor por defecto. `docker-compose.yml` traía `MSSQL_SA_PASSWORD:-PredictIT_2026!`, que servía para levantar la demostración con un comando y era un problema si alguien desplegaba ese archivo tal cual. Ahora usa la forma

`${MSSQL_SA_PASSWORD:?...}`: sin la variable, el stack no levanta y dice qué falta. Lo mismo con `PREDICTIT_DB_PASSWORD`.

2 · La aplicación ya no se conecta como `sa`. El script `db/06-usuario-de-aplicacion.sql` crea el login `predictit_app`, que es `db_owner` de las dos bases del sistema y no tiene ningún otro permiso sobre ellas ni sobre la instancia. Se comprobó: lee y escribe las dos bases, hace `BACKUP DATABASE`, y `CREATE DATABASE` le da *permission denied*.

Es `db_owner` y no `db_datareader` + `db_datawriter` porque el servicio de respaldos usa `BACKUP`, `RESTORE ... WITH REPLACE` y `ALTER DATABASE ... SET SINGLE_USER`, y las tres piden ser dueño de la base.

Lleva además el rol de servidor `dbcreator`, y esa es la concesión que hay que declarar. Restaurar es una operación de servidor: SQL Server la trata como crear la base de nuevo y pide `CREATE DATABASE` en `master` aunque la base ya exista. Sin ese rol, hasta `RESTORE VERIFYONLY` falla. La alternativa era sacar la restauración de la aplicación y dejarla como tarea del administrador de base; no se hizo porque respaldar y restaurar es un requisito del sistema (Req. Arq. 003).

Lo que `dbcreator` habilita de más es crear y borrar bases en la instancia. Lo que sigue sin poder hacer, que es la diferencia que importa frente a `sa`: entrar a una base ajena que no haya creado, crear o modificar logins, leer las credenciales cifradas de otra aplicación, cambiar la configuración del motor o apagarlo.

Las pruebas también corren como `predictit_app`. Corriendo como `sa`, un permiso que la aplicación no tenga pasa desapercibido y aparece en producción. Las 403 pruebas pasan con el usuario restringido.

3 · Cabeceras de seguridad en los dos lados. En la API las pone el middleware `CabecerasDeSeguridad`, y en el frontend `nginx` las incluye desde `cabeceras-de-seguridad.conf`:

Cabecera	Valor
`Content-Security-Policy`	`default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; …`
`X-Content-Type-Options`	`nosniff`
`X-Frame-Options`	`DENY`
`Referrer-Policy`	`strict-origin-when-cross-origin`
`Permissions-Policy`	`camera=(), microphone=(), geolocation=()`

Dos detalles que no son obvios:

- **`'unsafe-inline'` está en los estilos y no en los scripts.** Los temas se aplican escribiendo variables CSS en el elemento raíz, y eso son estilos en línea. En los scripts no hace falta, así que no está: sin esa excepción, un XSS no puede ejecutar lo que inyecte.
- **Las cabeceras están en un archivo incluido y no escritas en el `server`.** nginx no las hereda: un `location` que declare aunque sea un `add_header` propio pierde todos los de arriba. `/assets/` declara el suyo de caché, así que sin el include los archivos estáticos salían sin ninguna.

Qué falta. La API expone Swagger en el contenedor de demostración, y por eso su política de contenido admite estilos y scripts en línea —los que inyecta Swagger—. Apagando `Swagger__Habilitado` la política pasa sola a `default-src 'none'`, que es lo que corresponde a una API que sólo devuelve JSON.

A06 · Componentes vulnerables

Qué hace el sistema. Las versiones de todas las dependencias están fijadas, en `.csproj` y en `package-lock.json`. En el frontend el CI usa `npm ci` y no `npm install`: instala exactamente el lock y falla si no coincide, así que el CI prueba lo mismo que corre en la máquina del autor.

Fijar las versiones no alcanza —una versión fijada también puede tener un CVE publicado la semana que viene— así que el CI tiene un job **`dependencias`** que corre `dotnet list package --vulnerable --include-transitive` y `npm audit`, y deja el informe en el resumen del workflow.

Ese job **no corta la construcción**, avisa. Una vulnerabilidad en una dependencia transitiva puede aparecer un martes sin que nadie haya tocado el proyecto, y bloquear la entrega por eso enseña a ignorar el rojo, que es peor que no tener el control.

Qué falta. Dependabot, para que además de avisar proponga la actualización. Y que alguien mire el resumen: un control que informa y que nadie lee no es un control. Eso último no lo resuelve una herramienta.

Y pasó. El 13/10 se publicó la CVE-2026-84008 —denegación de servicio, severidad alta— sobre `System.Security.Cryptography.Pkcs` 9.0.13, que trae `Microsoft.Data.SqlClient`. Como el job avisa y no corta, apareció recién el 23/10, revisando la segunda entrega contra el código. Se fijó la 9.0.21. En el frontend había ocho más, todas en dependencias de desarrollo —Vite, Vitest y Puppeteer, que no llegan al sitio publicado—, y se actualizaron dentro de sus rangos. Hoy los dos informes dan cero.

A07 · Fallas de identificación y autenticación — parcial

Qué hace el sistema. Bloqueo de la cuenta a los **cinco intentos fallidos**, con desbloqueo por un administrador (`BloqueoPorIntentosTests`). Desde H-45, además, **límite de diez intentos por minuto y por dirección de origen**: el bloqueo por cuenta no ve el ataque que prueba una contraseña común sobre muchos usuarios, porque ahí nunca se juntan cinco intentos en la misma cuenta. El rechazo se asienta.

El token vence a las 8 horas y **no lleva las patentes adentro**: se consultan en cada pedido, así que revocar un permiso tiene efecto en la llamada siguiente y no al vencer la sesión. El token se guarda en `sessionStorage` y no en `localStorage`: al cerrar la pestaña la sesión se termina, que es lo esperable en una máquina compartida.

Qué falta. No hay segundo factor. Es la ausencia más significativa de la decisión del ADR 0014, y está declarada ahí. Tampoco hay política de complejidad de contraseña configurable ni recuperación por correo: el restablecimiento lo hace un administrador.

A08 · Fallas de integridad de datos y software

Qué hace el sistema. La **bitácora no se puede alterar**: un disparador `INSTEAD OF UPDATE, DELETE` rechaza cualquier intento con el error 50001, incluso conectándose directo a la base con `sqlcmd`. No es una convención que el código respete: es imposible de saltear.

Una consecuencia que apareció midiendo la carga (`CARGA.md`): **un usuario que hizo algo alguna vez no se puede eliminar**, porque la bitácora lo referencia. Por eso el sistema ofrece dar de baja y no borrar.

La deserialización se limita a JSON con tipos declarados (`System.Text.Json` con tipos concretos), sin deserialización polimórfica de tipos que vengan del pedido.

En las dependencias. El árbol completo de paquetes está congelado con su hash: cada proyecto de .NET tiene su `packages.lock.json` con las directas y las transitivas, y el frontend su `package-lock.json`. La integración continua restaura en **modo bloqueado** —`dotnet restore --locked-mode` y `npm ci`—, que falla si lo resuelto no coincide con el bloqueo en lugar de actualizarlo en silencio. La imagen de la API también: el `Dockerfile` copia los archivos de bloqueo antes de restaurar y usa el mismo modo, así que no puede construirse con un conjunto de paquetes distinto del que se probó. Actualizar una dependencia pasa a ser un cambio visible en el repositorio, que es el punto.

Además, el trabajo de la integración continua corre `dotnet list package --vulnerable --include-transitive` y `npm audit` en cada corrida, y publica el resultado en el resumen.

Qué falta. Nada en los datos. En la cadena de construcción queda la **firma de los artefactos**: las imágenes de contenedor no se firman y no hay attestation de procedencia. Está fuera del alcance declarado, y es lo que se haría con Sigstore o con la attestation de GitHub en un despliegue real.

A09 · Fallas de registro y monitoreo

Qué hace el sistema. Dos registros, separados a propósito. La **bitácora** es de negocio y auditoría —quién hizo qué—, y el **logger** es técnico, para lo que le pasa al sistema, incluido el caso en que la bitácora misma falla, que por definición no se puede asentar en la bitácora.

La bitácora asienta el inicio y el cierre de sesión, los intentos fallidos, el rechazo por exceso de intentos, y cada operación de negocio con su usuario, su organización y la entidad afectada. Hay una pantalla para consultarla con filtros y otra para los errores del sistema con su traza.

Un defecto propio de esta categoría es el que más caro salió en el proyecto: **ocho de nueve códigos de evento no estaban en el catálogo y el servicio los descartaba en silencio**, así que la bitácora tenía 37 inicios de sesión y cero eventos de negocio. Se encontró contando filas, no leyendo código. Hoy hay dos pruebas que impiden que vuelva a pasar.

Qué falta. No hay alertas: nadie recibe un aviso cuando aparecen cien rechazos por intentos en un minuto. Se ve entrando a la pantalla.

A10 · Falsificación de peticiones del lado del servidor (SSRF)

Qué hace el sistema. La única llamada saliente es al proveedor de IA, y su dirección **no es configurable por el usuario ni por la organización**: es una constante en `ProveedorIAClaude` (`https://api.anthropic.com/v1/messages`). Lo que la organización configura es el proveedor —`SIMULADO` o `CLAUDE`—, la clave y qué datos autoriza a enviar; nunca la URL.

Es la decisión que cierra la categoría. Si la dirección fuera un campo de la configuración, cualquier administrador de cualquier organización podría hacer que el servidor consultara una dirección interna de la red.

Qué falta. Nada identificado.

Lo que esta revisión no es

Una revisión leída, no una prueba de penetración. Verifica que los controles existan y estén donde dicen estar, con el archivo a la vista. **No prueba que no se puedan saltear**: para eso hace falta que alguien los ataque, y eso no se hizo.

Decirlo importa porque las dos cosas se confunden. Que A01 figure como «cubierto» significa que el control de acceso está implementado en dos capas y verificado por pruebas automatizadas, no que sea inviolable.

Fuentes

OWASP Foundation. (2021). *OWASP top 10: 2021*. https://owasp.org/Top10/

National Institute of Standards and Technology. (2017). *Digital identity guidelines: Authentication and lifecycle management* (NIST Special Publication 800-63B). https://pages.nist.gov/800-63-3/sp800-63b.html

Cobertura de pruebas

> *PredictIT · Trabajo Final de Ingeniería · Patricio Pettini · B00072710-T1*

Se genera corriendo la suite con coverlet. No se escribe a mano: un porcentaje escrito a mano sigue diciendo lo mismo el día que la cobertura se cae.

Medición

Fecha	15/09/2026 08:59
Líneas medidas	14738
Cobertura total	**52.1 %**

Por proyecto

Tabla 1. *Cobertura de líneas por proyecto*

Proyecto	Cobertura	Contra la línea de base
PredictIT.Api	52.8 %	+1.2
PredictIT.BLL	33.1 %	+4.0
PredictIT.DAO	64.7 %	-2.6
PredictIT.Domain	78.5 %	-1.3
PredictIT.Service	66.2 %	+1.8

Para qué sirve este número

Para una sola cosa: que la cobertura no baje sin que nadie lo note. La línea de base está en `docs/tecnico/_cobertura-base.json` y el script avisa cuando la medición actual queda más de 2 puntos por debajo.

No hay un objetivo de porcentaje declarado, y es deliberado. Hay código que no vale la pena cubrir y código que con el 100 % sigue estando mal. Poner un número como meta lleva a escribir pruebas para el número en lugar de para el riesgo, que es lo contrario de lo que se busca.

Lo que sí está declarado es qué tiene que estar cubierto: los catorce casos de prueba del capítulo 10, y eso se verifica aparte en `CASOS-DE-PRUEBA.md`.

Medición del requisito de carga

> *PredictIT · Trabajo Final de Ingeniería · Patricio Pettini · B00072710-T1*

El ADR 0008 compromete **500 activos y 5000 incidencias con respuesta por debajo de los 2000 ms**, y **20 usuarios concurrentes con p95 por debajo de ese mismo número**. Este documento es la medición de las dos cosas, y se genera corriendo `scripts/medir-carga.py`.

Cómo se midió

Se crea una organización aparte con el volumen que pide el ADR, se entra con un usuario de esa organización y se cronometran los endpoints que recorren tablas grandes. Al terminar se borra.

La organización es aparte de la de demostración por dos motivos: no ensuciar los datos que se muestran en la defensa, y porque el aislamiento por organización se aplica en el acceso a datos, así que medir contra una organización cargada es además la prueba de que ese filtro escala.

De cada endpoint se descarta la primera llamada —paga la compilación del plan de consulta y el arranque en frío del pool— y de las cinco siguientes se informa la mediana y la peor.

Resultado

Fecha	04/09/2026 11:42
Equipos	500
Incidencias	5000
Objetivo	2000 ms
Peor medición	84 ms
Cumple	**sí**

Tabla 2. *Tiempo de respuesta por endpoint, con el volumen del ADR 0008*

Qué se pidió	Endpoint	Código	Mediana	Peor
Listado de activos, primera página	`/api/equipos?porPagina=25`	200	26 ms	30 ms
Listado de activos, 500 de una	`/api/equipos?porPagina=1000`	200	19 ms	37 ms
Listado de incidencias, primera página	`/api/incidencias?porPagina=25`	200	30 ms	43 ms

Qué se pidió	Endpoint	Código	Mediana	Peor
Listado de incidencias, 1.000 de una	`/api/incidencias?porPagina=1000`	200	25 ms	43 ms
Tablero	`/api/prediccion/dashboard`	200	45 ms	51 ms
Panel predictivo	`/api/prediccion/panel`	200	25 ms	33 ms
Historial del parque	`/api/historial`	200	77 ms	84 ms

Lo que apareció al limpiar

La primera versión de la medición terminaba borrando el usuario que había usado, y el borrado falló: la bitácora lo referencia por clave foránea, y la bitácora no se puede borrar porque un disparador rechaza el DELETE.

No es un defecto: es la consecuencia buscada de tener un registro de auditoría inmutable. Un usuario que hizo algo alguna vez no se puede eliminar del sistema, y por eso el sistema ofrece **dar de baja** y no borrar. La medición ahora desactiva el usuario en lugar de borrarlo.

Concurrencia

El ADR 0008 compromete además **20 usuarios concurrentes con p95 por debajo de los 2000 ms**. Con un solo cliente no se ve la contención del pool de conexiones ni la cola del servidor, que es donde aparece el problema cuando aparece.

Cada usuario simulado recorre cuatro pantallas —tablero, activos, incidencias y análisis— en lugar de golpear un endpoint solo, y el barrido se repite 3 veces. Se informa el p95 y no el promedio: un promedio de 200 ms con una cola de cuatro segundos esconde justamente el caso que importa.

Tabla 3. *Tiempo de respuesta con 20 usuarios concurrentes*

Usuarios simultáneos	20
Peticiones	240
Mediana	61 ms
p95	**144 ms**
Peor	227 ms
Respuestas con error	0
Cumple	**sí**

Qué NO mide esto

Una base local sin latencia de red, en la misma máquina que la API. No mide el efecto de una base en otro servidor, ni una carga sostenida de horas, ni escrituras concurrentes sobre la misma incidencia.

Decirlo importa: se puede afirmar que el volumen y los veinte usuarios del ADR se cumplen en esta configuración, y no se puede afirmar nada sobre un despliegue distribuido.

Estabilidad bajo carga sostenida

PredictIT · Trabajo Final de Ingeniería · Patricio Pettini · B00072710-T1

RNF-09 pide que el sistema **no muestre crecimiento sostenido de memoria ni agote el pool de conexiones** durante la ejecución prolongada del motor predictivo sobre el parque completo. Este documento es esa medición, y se genera corriendo `scripts/medir-estabilidad.py`.

Cómo se midió

Sobre la organización de medición de `CARGA.md` —500 equipos y 5000 incidencias—, con las 5 reglas del Estudio copiadas, porque las reglas son de cada organización y sin ellas el motor no evalúa nada. Se descartan 5 evaluaciones de calentamiento y después se evalúa el parque completo 120 veces seguidas. Después de cada vuelta se toma la memoria del contenedor de la API y las sesiones abiertas contra SQL Server por la cuenta de la aplicación, agrupadas por nombre de aplicación: `PredictIT-Fondo` es el pool propio de los procesos de fondo (ADR 0009).

El criterio. Hay crecimiento sostenido si la tendencia de la segunda mitad de la corrida, proyectada sobre esa misma mitad, suma más de un 10 % de su memoria media. «Sostenido» quiere decir que no se detiene: un proceso que sube mientras se calienta y después queda plano no está perdiendo memoria, y uno que pierde sigue subiendo en proporción al trabajo, también al final. Se informan además el piso de cada tercio y la mediana de los extremos, que incluyen el calentamiento.

El criterio cambió dos veces, y las dos corridas anteriores quedan escritas porque cambiar un criterio después de ver un resultado hay que poder justificarlo. La primera —60 vueltas, el 23/10 a las 20:37— comparaba la mediana de las primeras diez vueltas contra la de las últimas diez y dio +13,2 %: «no cumple». La serie no era una recta —75 MiB en la vuelta 30, 66 en la 40, 75 en la 60—, así que se pasó a comparar el piso de cada tercio y se duplicó la corrida. La segunda —120 vueltas, a las 20:58— dio +20,4 % en el piso: «no cumple» otra vez. Pero esa serie mostraba lo que pasaba: la memoria subió de 65 a 78 MiB en las primeras cincuenta y tantas vueltas y en las sesenta y cinco siguientes no se movió de 78-79, con una pendiente de +0,01 MiB por vuelta. Los dos criterios comparaban el final contra el calentamiento del montón. El que responde la pregunta de RNF-09 —si el crecimiento se sostiene— es la tendencia una vez estabilizado, y es el que se usa.

Resultado

Fecha	23/10/2026 21:20
Evaluaciones del parque completo	120, más 5 de calentamiento
Primera evaluación, en frío	11.5 s, 500 equipos evaluados y 321 alertas nuevas
Tiempo por evaluación	mediana 5.4 s
Tiempo por evaluación, con el barrido de fondo corriendo a la vez	mediana 5.7 s en 38 vueltas
Tiempo por evaluación, sin el barrido	mediana 5.3 s en 82 vueltas
La evaluación más lenta	54.8 s, en la vuelta 99, sin el barrido de fondo
Evaluaciones que tardaron más del doble de la mediana	5 de 120: vuelta 18, 30 s, vuelta 40, 31 s, vuelta 55, 30 s, vuelta 78, 42 s, vuelta 99, 55 s
Vueltas que no evaluaron los 500 equipos	0
Memoria de la API, rango	de 64.1 a 77.6 MiB
Pendiente en la segunda mitad	+0.035 MiB por vuelta
Crecimiento proyectado en la segunda mitad	**+3.1 %**
Piso del primer y del último tercio, con el calentamiento	64.1 y 67.9 MiB (+6.0 %)
Mediana de las primeras y las últimas 10 vueltas, con el calentamiento	65.9 y 69.7 MiB (+5.7 %)
Sesiones abiertas como máximo, `Core Microsoft SqlClient Data Provider`	2
Sesiones abiertas como máximo, `PredictIT-Fondo`	2
Respuestas con error	0
Cumple	**sí**

Tabla 4. *Memoria y conexiones cada diez vueltas*

Vuelta	Evaluación	Memoria de la API	Sesiones por aplicación
1	6.6 s	64.1 MiB	Core Microsoft SqlClient Data Provider: 2
10	5.2 s	69.3 MiB	Core Microsoft SqlClient Data Provider: 2
20	5.9 s	73.6 MiB	Core Microsoft SqlClient Data Provider: 2, PredictIT-Fondo: 2
30	5.4 s	75.5 MiB	Core Microsoft SqlClient Data Provider: 2, PredictIT-Fondo: 2
40	30.7 s	65.7 MiB	Core Microsoft SqlClient Data Provider: 2, PredictIT-Fondo: 2
50	7.4 s	66.7 MiB	Core Microsoft SqlClient Data Provider: 2
60	5.7 s	68.9 MiB	Core Microsoft SqlClient Data Provider: 2

Vuelta	Evaluación	Memoria de la API	Sesiones por aplicación
70	5.2 s	67.9 MiB	Core Microsoft SqlClient Data Provider: 2
80	5.7 s	69.0 MiB	Core Microsoft SqlClient Data Provider: 2
90	4.5 s	68.0 MiB	Core Microsoft SqlClient Data Provider: 2
100	4.8 s	70.2 MiB	Core Microsoft SqlClient Data Provider: 2
110	4.5 s	69.7 MiB	Core Microsoft SqlClient Data Provider: 2
120	4.2 s	69.6 MiB	Core Microsoft SqlClient Data Provider: 2

La serie completa de la memoria, vuelta por vuelta, en MiB: 64, 64, 65, 65, 66, 66, 67, 69, 69, 69, 71, 70, 69, 70, 70, 70, 71, 73, 73, 74, 73, 74, 74, 74, 75, 75, 76, 77, 78, 75, 66, 65, 65, 66, 66, 65, 65, 66, 66, 66, 66, 67, 66, 66, 67, 67, 66, 66, 68, 67, 67, 68, 67, 67, 69, 72, 69, 69, 72, 69, 68, 69, 68, 68, 68, 69, 68, 68, 69, 68, 68, 69, 68, 68, 69, 68, 68, 68, 68, 69, 68, 68, 69, 68, 68, 69, 68, 68, 69, 68, 68, 69, 68, 68, 68, 68, 69, 68, 69, 70, 69, 69, 70, 70, 70, 70, 70, 70, 69, 70, 70, 70, 70, 70, 70, 70, 69, 69, 70, 70.

Lo que no mide

Es una corrida de minutos, no de días: un crecimiento muy lento puede no aparecer en 120 vueltas. Mide la evaluación pedida por la API, que usa el pool de las peticiones; el barrido programado usa el suyo (ADR 0009), que aparece en la tabla sólo si coincidió con la corrida. Y mide la memoria del contenedor, que incluye lo que el recolector reservó y todavía no devolvió: es una cota superior de lo que la aplicación usa, no lo que usa.

Y no explica las evaluaciones lentas. Con una mediana de cinco segundos, algunas tardan entre treinta y sesenta, con el barrido de fondo corriendo y sin él. No se diagnosticó la causa: los candidatos son la actualización automática de estadísticas de SQL Server sobre tablas que crecen de a quinientas filas por vuelta y la contención del propio equipo donde corre. Queda anotado en `docs/PENDIENTE.md`.

Auditoría de uso del sistema

> *PredictIT · Trabajo Final de Ingeniería · Patricio Pettini · B00072710-T1*

Recorrido completo del sistema andando, con los cuatro perfiles, midiendo lo que se puede medir y anotando lo que hay que mirar. No es una revisión del código: es la revisión de lo que ve y hace una persona.

Cómo se hizo

El sistema levantado de punta a punta —SQL Server en contenedor, API, frontend— y sobre eso:

1. **Matriz de permisos.** Cada uno de los cuatro perfiles contra 19 endpoints de lectura y 6 operaciones de escritura, comparando lo que devuelve contra lo que debería.
2. **Recorrido por pantalla.** Login, las cuatro pantallas del solicitante y las once del administrador, con cada perfil por separado.
3. **Un flujo real de punta a punta.** Reportar una incidencia como solicitante, ver qué hace el sistema con ella, seguirla hasta el detalle.
4. **Mediciones.** Contraste de color, ancho útil en pantalla chica, cantidad de alertas contra tamaño del parque, tiempo de respuesta.
5. **Despliegue.** Las dos imágenes de contenedor construidas desde cero.

Veredicto

El sistema **funciona de punta a punta** y hace lo que promete. El aislamiento entre organizaciones, los permisos por perfil, la clasificación automática, la asignación con justificación y la bitácora inmutable: los cinco se comprobaron andando, no leídos del código.

Aparecieron **tres defectos**, los tres corregidos y verificados durante la auditoría. Uno era bloqueante: el usuario solicitante no podía usar la única función que tiene.

Quedan **siete observaciones abiertas**, ninguna bloqueante. La más importante no es un error sino una decisión de producto: el sistema genera más alertas de las que una persona puede atender.

Lo que está bien, con la evidencia

La trazabilidad de la decisión automática

Es lo mejor del sistema y es exactamente lo que el trabajo promete. Se cargó una incidencia real desde la pantalla del solicitante:

> *«La notebook se apaga sola cuando la muevo. Desde el lunes. Si la muevo de lugar mientras la uso se apaga de golpe, sin aviso. Probé con otro cargador y pasa igual.»*

Y el sistema respondió, en la misma pantalla:

> ***Incidencia #029 registrada*** *· Clasificación:* ***Hardware*** *· Asignada a:*
>
> ***Martín Acosta***
>
> *Por qué: *«Seleccionado por especialidad «Hardware» nivel 3; 5 incidencias abiertas; ya resolvió un caso de este equipo.»**

De texto libre a una decisión con tres motivos nombrados. Un sistema que asigna sin decir por qué obliga a confiar; éste se deja auditar. En una incidencia anterior, además, detectó que duplicaba a otra y lo dijo.

El aislamiento entre organizaciones

Verificado cambiando de organización con el perfil Partner: de «Estudio Pettini & Asoc.» (52 equipos, 29 incidencias) a «Clínica San Isidro» (2 equipos, 0 incidencias). **Ningún equipo aparece en las dos.** El filtro se aplica en el acceso a datos, no en la pantalla, así que no hay forma de saltearlo desde el navegador.

Los permisos

Los cuatro perfiles contra 25 operaciones. Todo dio lo esperado:

	Solicitante	Técnico	Administrador	Partner
Reportar una incidencia	sí	sí	sí	sí
Ver el inventario	**no**	sí	sí	sí
Crear un equipo	**no**	**no**	sí	sí
Reevaluar el riesgo	**no**	sí	sí	sí
Respaldar la base	**no**	**no**	sí	sí

	Solicitante	Técnico	Administrador	Partner
Ver usuarios y bitácora	**no**	**no**	sí	sí
Cambiar de organización	**no**	**no**	**no**	sí

El Partner tiene una patente más que el administrador —cambiar de organización— y es correcto: es el modelo de revendedor multi-cliente del plan de negocio.

El contraste

Todo por encima del mínimo de accesibilidad (4,5:1):

Elemento	Contraste	Exigido
Botón principal	**7,36:1**	4,5:1
Texto secundario	5,09:1	4,5:1
Etiqueta de riesgo alto	6,47:1	4,5:1
Etiqueta de riesgo medio	5,38:1	4,5:1
Encabezado de tabla	5,09:1	4,5:1

Y el riesgo se muestra con **barra, número y palabra** a la vez, así que no depende del color: quien no distingue rojo de verde lee «97 ALTO» igual.

La bitácora

Registró las acciones de esta misma auditoría mientras ocurrían: «admin generó el reporte «parque» con 52 registros», «Incidencia #29 clasificada automáticamente (HEURISTICA): Hardware». 359 eventos, filtrables por fecha, tipo, usuario y criticidad.

Los reportes

Los tres generan PDF real —67 KB, firma `%PDF-`, tipo correcto— y cada uno declara **qué pregunta responde** en lugar de sólo cómo se llama. «Qué equipos hay, dónde están, quién responde por cada uno y cuál está en riesgo» dice más que «Reporte de parque».

El despliegue

Las dos imágenes de contenedor construyen desde cero sin intervención.

Defectos encontrados y corregidos

1 · El solicitante no podía reportar nada · bloqueante

El formulario de alta pedía el inventario completo (`GET /api/equipos`), que ese perfil no tiene permiso de ver. Recibía **403**, el desplegable de equipos quedaba **vacío** y el error se tragaba en silencio: una pantalla que no funcionaba, sin decir por qué.

La única función del perfil era imposible.

Corregido: quien ve el inventario elige de ahí, quien no, elige entre los equipos de los que es responsable. Y si falla, ahora lo dice.

2 · Lo primero que veía el solicitante era un error de permiso

Al iniciar sesión, el comodín de rutas mandaba a todos a `/activos`. Ese perfil no tiene permiso para esa pantalla, así que la primera impresión del sistema era un cartel rojo sobre una tabla vacía.

Corregido: se calcula la primera pantalla que el perfil puede ver, tomando el primer ítem de su propio menú. Cualquier perfil nuevo queda cubierto sin tocar nada.

3 · La consola se llenaba de un error de React en cada pantalla

`Cannot update a component while rendering a different one`. La función de traducción anotaba las claves faltantes con un cambio de estado **durante el render**. Es el tipo de cosa que hoy es un aviso y en la próxima versión de React es un error.

Corregido: la anotación se encola para el turno siguiente. Verificado: cero apariciones en el arranque limpio.

Observaciones abiertas

A · El sistema alerta sobre el 87 % del parque · la más importante

Medido: **60 alertas activas sobre 45 de los 52 equipos**. De esas 60, sólo 15 superan los 65 puntos de riesgo; **44 están por debajo de 30**.

El tablero muestra dos números contradictorios uno al lado del otro:

> *Alertas predictivas: **60** · Equipos en riesgo alto: **3***

El primero dice «hay sesenta cosas que hacer» y el segundo dice la verdad. La pantalla de análisis sí lo enmarca bien —«riesgo alto 3 equipos, 6,4 %; medio 7; bajo 37»— pero el tablero, que es lo primero que se ve, no usa ese marco.

Un sistema que marca casi todo el parque no ayuda a priorizar: devuelve el problema que venía a resolver.

Qué haría: que el indicador del tablero cuente lo accionable y no el total, y que el panel de alertas se pueda filtrar por nivel. Hoy no tiene **ningún filtro**: son 60 tarjetas para recorrer a mano, cuatro de ellas del mismo equipo.

B · No se puede usar en un teléfono

Medido a 375 px de ancho: la barra lateral ocupa **235 px, el 63 % de la pantalla**, y la tabla de 793 px queda dentro de los 140 restantes. La única media query de toda la hoja de estilos es `prefers-reduced-motion`.

Para un sistema de mantenimiento pesa más de lo que parece: el técnico está parado frente al equipo roto, con el teléfono en la mano.

C · Quien reporta no puede hacer nada más

El solicitante carga la incidencia y a partir de ahí es espectador: no puede agregar un dato, ni responder una pregunta del técnico, ni confirmar que quedó resuelto. En un sistema de tickets, el que reportó es parte de la conversación.

D · El solicitante ve la consola del técnico

El módulo del cliente reusa la pantalla de incidencias del administrador, con las columnas «Técnico», «Asignación» (IA / Automática / Manual) y «Antigüedad». Son datos de operación interna. El capítulo 10.5.2.3 del documento llama a esa pantalla «Mis tickets»; el sistema la titula «Incidencias» y no dice que sean las suyas.

E · Las explicaciones están después de lo que explican

Se repite en tres pantallas: la nota que dice por qué la pantalla es como es aparece **debajo** de la tabla. En «Usuarios», la explicación de por qué no hay botón de alta está al final, después de que el administrador ya lo buscó. En «Reportes» y en «Bitácora», en cambio, está arriba —y ahí funciona—.

F · El texto es chico y la densidad no lo cambia

Encabezados de tabla en **11 px**, texto secundario y etiquetas en 11,5 px. La preferencia de densidad (Compacta / Normal / Amplia) mueve el espaciado y el alto de fila —36, 44, 52 px—

pero **no el tamaño del texto**. Quien necesita letra más grande no tiene control, sólo el zoom del navegador.

G · Detalles menores

- La columna «Origen» de la bitácora muestra `::1`. Es correcto y no significa nada para quien lo lee.
- En el tablero del técnico, «Motivo principal» —la columna que explica— se corta.
- En el tablero, «Registrar incidencia» es el botón principal aunque el técnico atiende y no reporta.

La prueba del chico de 15 años

Se recorrió el sistema preguntando, en cada pantalla, si alguien sin formación técnica entiende qué hace y qué se espera de él.

Pasa bien:

- El formulario de incidencia. La sección opcional dice «Opcional» y explica por qué se puede dejar vacía: «Si no sabés qué elegir, dejalo vacío: el sistema lo sugiere a partir de lo que escribiste». Los ejemplos están en el campo, no en un instructivo.
- Los estados vacíos. «Este equipo no tiene incidencias ni mantenimientos registrados» y «No tenés permiso para ver el historial de este equipo» son dos cosas distintas y el sistema las distingue.
- Las secciones fuera de alcance. En vez de un «no implementado», explican qué harían, por qué no están y qué haría falta. Un interruptor que no hace nada es una mentira; esto es lo contrario.
- Las alertas traen la recomendación al lado del dato: «4 incidencias de la categoría Hardware en los últimos 90 días (umbral: 2) a La falla se repite: revisar si la reparación anterior atacó la causa».

No pasa:

- «Confianza 20 %» en la clasificación sugerida. Es honesto y no significa nada para quien no sabe qué se está midiendo.
- «Score 97» sin decir de qué. La pantalla de análisis lo explica; el tablero, donde aparece primero, no.
- La columna «Permisos: 12» en el listado de usuarios: un número sin unidad ni comparación.

Valor para el cliente

Lo que un cliente estaría pagando, en orden:

1. **Saber por qué.** Cada asignación y cada alerta dicen su motivo con nombres y números. Eso es lo que separa esto de una caja negra, y es defendible frente a alguien que pregunte «¿por qué me lo asignaron a mí?».
2. **La bitácora inmutable.** No la protege la pantalla: la protege un disparador que rechaza el borrado incluso conectándose directo a la base. Es una afirmación verificable, no una promesa.
3. **El historial por equipo.** Es lo que hace posible el análisis y lo que se pierde si se pierde la base. El sistema lo dice en la pantalla de respaldos, con esas palabras.
4. **El multi-cliente real.** Comprobado, no declarado.

Lo que hoy **no** está dando el valor que podría: el motor predictivo. La maquinaria está —cinco reglas, ponderación, calibración, umbrales configurables— y el resultado es una lista de 60 alertas que nadie va a recorrer. El problema no es el motor: es que la salida no está priorizada.

Qué haría primero

En orden de relación entre lo que cuesta y lo que cambia:

1. **Priorizar la salida del motor** (observación A). Un filtro por nivel en el panel y que el indicador del tablero cuente lo accionable. Es media jornada y convierte la función central de «interesante» en «usable».
2. **Mover las tres explicaciones arriba de lo que explican** (E). Es una hora.
3. **Dejar que quien reportó pueda comentar** (C). Cierra el circuito del ticket.
4. **Responsive** (B). Es el más caro de los cuatro y el que más cambia dónde se puede usar el sistema.

_{Auditoría hecha sobre el sistema andando. Los tres defectos de la sección anterior están corregidos y verificados; las observaciones quedan abiertas.}

Revisión de coherencia entre los documentos y el sistema

> *PredictIT · Trabajo Final de Ingeniería · Patricio Pettini · B00072710-T1 ·*
>
> ***Grupo 2** · Universidad Abierta Interamericana*
>
> *Revisión del **09/09/2026** sobre el paquete completo: el documento del TFI, el presupuesto financiero, la Entrega 1 de negocio, los cinco manuales, la documentación técnica, la gestión del proyecto y el sistema andando.*

Qué se buscó

Una sola cosa, en todas partes: **dónde un documento dice algo que otro documento, o el sistema, desmiente**. No se revisó si las decisiones son buenas —eso es materia de la defensa— sino si lo que el paquete afirma sobre sí mismo se sostiene.

El criterio para dar por buena una afirmación fue medirla, no leerla. Cuando el documento dice «257 pruebas», se corrió la suite; cuando dice «la interfaz es multiidioma», se abrió la aplicación en inglés; cuando dice «el punto de equilibrio está en 135 clientes», se recalculó el libro.

Resumen

Hallazgos	18
Corregidos	18
Controles automáticos nuevos	5
Controles de alineación en total	17

Ningún hallazgo es de criterio ni de opinión: los dieciocho son afirmaciones verificables que no se cumplían.

1 · La sección financiera se contradecía a sí misma

Gravedad: alta. Es el hallazgo que un evaluador puede encontrar solo, con una calculadora.

La sección 7.7 del documento tiene tres tablas —presupuesto mensual, ingresos y resultado, por etapa— y varios párrafos. Las tablas venían de la estructura de costos anterior a la recalibración D-33 y los párrafos, de la posterior:

Concepto, año 1	Tablas de 7.7	Prosa de 7.7 y modelo
Costos de recursos humanos	ARS 45,9 M	ARS 23,9 M
Egresos totales	ARS 53,6 M	ARS 32,4 M
Ingresos	ARS 6,1 M	ARS 10,4 M
Punto de equilibrio	—	135 clientes, mayo de 2029

Sumar los doce meses de la tabla y compararlos con el libro es todo lo que hacía falta para verlo. La nota estaba escrita —`docs/negocio/README.md` decía «la reconciliación ya no cierra… los dos números hay que rehacerlos en el documento»— y quedó pendiente tres iteraciones.

Corregido. Las tres tablas se rehicieron desde el presupuesto, promediando los meses de cada etapa, y la prosa que las describía se puso al día: el salto de personal entre E2 y E3 es del 62 % y no «por más de cinco». Se agregaron los totales anuales al pie, que es lo que ata las tablas al libro sin obligar a sumar. `docs/negocio/README.md` dejó de decir que no cierra, porque cierra.

2 · El desvío entre la meta y la proyección no estaba declarado

Gravedad: alta. El documento fija 25 clientes pagos al mes 12; el modelo proyecta 22,8, un 8,9 % por debajo. La Entrega 1 de negocio lo declara con su motivo —haber sacado la demostración comercial del embudo baja la conversión— y el documento del TFI no lo decía.

Corregido. La meta se sostiene como objetivo y la diferencia queda escrita en 7.7, que es distinto de presentar la proyección como si coincidiera.

3 · El multiidioma no cumplía lo que el requisito no funcional promete

Gravedad: alta. El RNF dice que «la incorporación de un nuevo idioma no requiere modificaciones en la lógica de la aplicación, sino únicamente la traducción del archivo correspondiente». No era cierto.

Con la interfaz en castellano todo se leía bien, y ese es exactamente el motivo por el que nadie lo vio. Puesta en inglés quedaban en castellano:

- los cuatro indicadores del tablero, que la capa de negocio armaba como frases («Equipos activos», «de 52 registrados»), de modo que sumar un idioma pedía editar `PrediccionBusiness`;

- unos noventa rótulos del frontend escritos como literales;
- treinta y cuatro claves que el código le pedía al diccionario y que nadie había cargado, que se ven idénticas a una traducida porque el texto de reserva está en castellano.

El control que vigilaba esto informaba cero. Su heurística exigía un acento, una eñe o un artículo, y «Estado operativo», «Fallos seguidos» o «Configurar reglas» no tienen ninguno.

Corregido. Los indicadores viajan como clave, con los números aparte —en inglés el número va delante del sustantivo y en castellano detrás, así que concatenar no alcanza—. La detección mira además los rótulos sin acento, las props propias que llevan texto visible y ya no descarta un párrafo por tener un punto y coma. **Red nueva:** `npm run idioma` abre el sistema en los dos idiomas, compara pantalla contra pantalla y descarta lo que mandó la API, que es la única forma de distinguir un texto sin traducir de un dato del catálogo. Da cero.

4 · Los manuales decían cuántas pruebas esperar, y no era ese número

Gravedad: media. Quien sigue el manual de instalación y ve otro número no sabe si le faltó un paso.

Documento	Decía	Es
Manual de instalación	131 backend, 24 frontend	257 y 34
Manual de programador	131	257
Patrones y principios	253	257
Revisión OWASP	197	257
README del repositorio	175 y 24	257 y 34
Insignia de la hoja de ruta	175 + 24	se cuenta sola

Corregido, y con el control 16, que compara cada número declarado contra el total de la corrida. Las cuentas históricas —el log de uso de IA, el registro de cambios— no se tocaron: ahí el número es el que había ese día.

5 · Nueve referencias cruzadas apuntaban a apartados que no existen

Gravedad: media. El capítulo 5 se renumeró de 5.3.x a 5.2.x y las menciones no se movieron con él: quedaron nueve, tres de ellas dentro de la sección financiera. Una mención rota no falla; manda al lector a buscar algo que no está.

Corregido, y con el control 15, que verifica que toda referencia a un apartado numerado apunte a un título que existe.

6 · El CP-15 no figuraba como caso de prueba

Gravedad: media. Su tabla estaba rotulada «Tabla 86. CP-14 (continuación)» y no tenía título propio, así que el único caso que el documento no lista como apartado era justamente el último que se agregó. En el índice, la especificación de casos de prueba terminaba en CP-14.

Corregido: `10.4.10.15` con su epígrafe.

7 · El apartado 7.7.1 estaba antes del cuerpo de 7.7

Gravedad: media. El apartado del valor terminal se había insertado justo después del título de la sección, de modo que el presupuesto, los ingresos, el resultado y el punto de equilibrio colgaban de un título que habla de otra cosa.

Corregido: el apartado pasa al final, que es además donde corresponde por

contenido: es la conclusión de la evaluación.

8 · Seis títulos vacíos

Gravedad: baja, pero visible: salen en el índice como renglones en blanco numerados. **Corregido.**

9 · Basura de codificación en el título de 6.6.2

Los apóstrofos de «7 C's» habían quedado como diéresis sueltas. **Corregido.**

10 · El plan de sprints y el backlog no coincidían

Gravedad: media. El plan describía ocho sprints y el backlog tiene diez. El décimo figuraba en el plan como colchón «sin alcance asignado» y tiene diecisiete historias terminadas.

Detrás había algo más de fondo: **el trabajo se adelantó al calendario**. Están terminadas todas las historias de S6, S7, S8 y S10 mientras el cronograma pone a S5 como sprint en curso.

Corregido diciéndolo, no corriendo las fechas: las de cada sprint son las que se planificaron en julio y los estados son lo que está hecho. Un cronograma que se reescribe cada vez que se cumple no sirve para medir nada. Se agregó la sección de S10 con lo que efectivamente absorbió.

11 · Una prueba enumeraba siete secciones de Configuración y hay nueve

Gravedad: baja. La prueba se llamaba «lista las siete secciones» y verificaba siete; se le habían escapado «Errores del sistema» y «Apariencia», que son las dos últimas que se agregaron. Una prueba que enumera de menos pasa igual. **Corregido.**

12 · Una prueba del frontend estaba en rojo en el repositorio

Gravedad: alta para la entrega. `Organizacion.test.tsx` fallaba con «`api.organizacion.consumoIa` no es una función»: el doble de la API se había quedado atrás de la pantalla, y los cinco casos del plan se caían por algo que no estaban probando. **Corregido**; la suite del frontend está en 34 de 34.

13 · Los marcadores del texto de reserva no se reemplazaban

Gravedad: baja, encontrado por el hallazgo 3. Cuando una pantalla se monta sin el proveedor de idioma, el traductor devolvía el castellano de reserva **sin** reemplazar los `{marcadores}`: «Con {equipos} equipos, el {plan} costaría {costo}». **Corregido.**

14 · El guion de la defensa daba respuestas que ya no son ciertas

Gravedad: alta. Son las frases que se dicen en voz alta delante del tribunal.

Pregunta	Decía	Es
¿El proyecto es viable económicamente?	«El VAN da −48 millones, la TIR 41 % nominal, y se vuelve positivo a 2,0×»	+184,7 M con valor terminal, −61,6 M sin él, TIR 113 %
¿Cuántos usuarios simultáneos aguanta?	«No lo sé, y no está medido»	Está medido: p95 de 144 ms con 20 concurrentes
¿Qué le falta al sistema?	«Cinco historias… el cuerpo de las pantallas todavía no sale del diccionario»	Dos, y las dos dependen de fechas

Corregido, incluida la diapositiva de viabilidad económica de la presentación y la de «lo que falta», que enumeraba cuatro pendientes de los cuales tres estaban resueltos —el multiidioma, la medición de carga y el límite de intentos por dirección de origen—.

15 · El panel de obra mostraba el proyecto de hace una semana

Gravedad: media. Es un entregable y se lee como un tablero medido: decía 71 de 76 historias, 227 pruebas, 32 correcciones al documento, y el diagrama de sprints mostraba S6 a S9 sin empezar con todas sus historias terminadas.

Corregido con `docs/panel/_actualizar-cifras.py`, que cuenta cada cifra de su fuente —el backlog, la corrida de la suite, el registro de cambios y el código— y redibuja las barras del diagrama. Lo que no se toca es el relato: la frase de los treinta defectos sale de las retrospectivas y no hay de dónde recalcularla, así que cambiar ese número sin fuente sería peor que dejarlo con su período declarado.

16 · El armado del paquete no se detenía con la suite en rojo

Gravedad: alta. El generador de la evidencia sale con código 0 aunque haya pruebas fallando —documenta las fallas en lugar de esconderlas, y para el documento está bien—, y el armado seguía adelante. Pasó de verdad durante esta misma revisión: sin `PREDICTIT_DB_PASSWORD` en el ambiente, las 104 pruebas que tocan la base fallaron y el paquete se armó igual, declarando 104 rojas.

Corregido: `_armar-entregables.py` lee el resultado de la corrida y se

detiene si hay una sola prueba en rojo, diciendo cuál es la causa más probable.

17 · La insignia de pruebas y el número de casos no se contaban

Gravedad: baja. «175 + 24» en la hoja de ruta y «los catorce casos tienen prueba automatizada» en la evidencia, cuando ya eran quince. **Corregido:** los dos números se cuentan.

18 · Los recuentos del README

Gravedad: baja, y es la primera página que abre cualquiera: decía 11 ADR cuando hay 14, 19 deltas cuando hay 43, 16 prompts críticos cuando son 18 y 53 historias cuando son 88. Nada los recalculaba.

Corregido, y con el control 17, que cuenta las cuatro cosas de su fuente.

Lo que se verificó y estaba bien

Vale decirlo, porque una revisión que sólo lista defectos no informa sobre el estado del trabajo:

- **Los precios** de los tres planes coinciden en el documento, el presupuesto, la Entrega 1 y el sistema, y no queda ningún precio viejo suelto.
- **Las fechas** de las tres entregas, y de qué grupo es cada una, coinciden en los cuatro lugares donde aparecen.
- **Los supuestos macroeconómicos** del REM coinciden en el documento, el libro y la Entrega 1, y los dos años extrapolados están declarados como tales en cada fila.
- **El dimensionamiento del mercado** cierra: 549.100 → 65.965 → 21.109, con los dos supuestos propios declarados y el rango de sensibilidad al lado.
- **El diccionario de datos** dice 22 tablas de negocio y la base tiene 22: se genera leyendo el esquema, no los scripts.
- **Los quince casos de prueba** del capítulo 10 tienen su correspondencia declarada con las pruebas que los ejercitan, y el CP-15 dice «12 pruebas» porque son doce.
- **Las veinticinco pantallas** del sistema están descritas en el capítulo 10.5 y en los manuales.
- **El stack** que declara el documento es el que está en el repositorio.
- **Ningún emoji ni flecha tipográfica** en los `.docx` ni en los PDF.

Los tres controles nuevos

Cada hallazgo que se podía volver a producir dejó un control, y cada control se probó **en rojo** antes de darlo por bueno:

	Qué verifica	Dónde
Control 15	Toda referencia a un apartado numerado apunta a un título que existe	`docs/_verificar-alineacion.py`
Control 16	La cantidad de pruebas que declara cada documento es la de la corrida	`docs/_verificar-alineacion.py`
`npm run idioma`	Ningún texto de interfaz queda en castellano con el sistema en inglés	`frontend/scripts/idioma-en-pantalla.mjs`
Control 17	Los recuentos del README —ADR, deltas, prompts críticos, historias— son los que hay	`docs/_verificar-alineacion.py`
Corte del armado	El paquete no se arma con una sola prueba en rojo	`_armar-entregables.py`

A los que se suma, del lado del sistema, una prueba de integración que exige que los indicadores del tablero viajen como clave del diccionario y no como frase armada.

Corrida de prueba: un clon limpio del repositorio

> *PredictIT · Trabajo Final de Ingeniería · Patricio Pettini · B00072710-T1*

10 de septiembre de 2026, de 17:05 a 17:30. Commit probado: a128171.

El criterio 3 de la historia H-53 —el congelamiento— pide que **un clon limpio del repositorio levante y pase todo**. Esta es una corrida de prueba de ese criterio, seis semanas antes del congelamiento del 30/10, y con un propósito concreto: descubrir ahora lo que falta en el repositorio en lugar de descubrirlo el día que ya no se puede tocar nada.

No reemplaza al congelamiento. Lo que verifica es que el contenido del repositorio alcance; la etiqueta de versión y el respaldo de la base de demostración siguen pendientes y son del 30/10.

Cómo se hizo

Un clon nuevo desde GitHub en una carpeta que no es la de trabajo, y a partir de ahí sólo lo que dice el README, sin copiar nada del checkout de desarrollo:

```
git clone https://github.com/PatricioPettini/trabajo-final-ingenier-a.git
cp .env.ejemplo .env        # y generar las dos claves como indica el archivo
docker compose up --build
```

Las contraseñas y las dos claves se generaron nuevas, distintas de las de la máquina de trabajo: si algo dependiera de un valor que sólo existe acá, tenía que aparecer.

Resultado

Paso	Resultado
`git clone`	**Falló la primera vez.** Ver el hallazgo 1
`docker compose up --build`	Las dos imágenes se construyen y los cuatro contenedores arrancan
Esquema y datos	El contenedor de migraciones aplica los seis scripts y sale en cero: 22 tablas de negocio, 15 de servicio, 739 claves de traducción y el usuario de aplicación creado
Aplicación	`http://localhost:8080` responde 200
API y Swagger	`http://localhost:8081/swagger` responde 200
Inicio de sesión	`admin` entra y recibe token
Datos de demostración	52 equipos, los mismos de la semilla
Tablero	Los cuatro indicadores llegan como clave del diccionario, no como frase
Análisis predictivo	A los dos minutos del arranque: 47 equipos evaluados, 59 alertas, 3 en riesgo alto, 5 medio y 39 bajo

Paso	Resultado
`dotnet test`	**250 de 257 la primera vez.** Ver el hallazgo 2. Con el arreglo, **257 en verde**
`tsc --noEmit`	Limpio
`npm test`	**34 en verde**
`npm run build`	Construye

Los tres hallazgos

1 · El repositorio no clonaba en Windows

```
error: unable to create file ENTREGABLES/ENTREGA FINAL/05 - Documentación
técnica/7 - Modelo Enterprise Architect (DER, casos de uso y clases).eap:
Filename too long
fatal: unable to checkout working tree
```

El clon se baja y el árbol de trabajo queda **sin escribir**: 522 archivos faltantes y un repositorio inservible. En Windows el límite de `MAX_PATH` son 260 caracteres y se reparten entre la carpeta destino y la ruta interna del archivo. La ruta interna más larga del repositorio eran 127 caracteres, y los sobrantes se los llevaba un solo archivo: el modelo de Enterprise Architect, que traía entre paréntesis lo que contiene.

Con 127 adentro, cualquiera que clonara en una carpeta de más de unos 130 caracteres se quedaba sin poder trabajar. No es un caso rebuscado: la carpeta de descargas de un usuario con nombre largo, o un `Documents\Facultad\TFI\ correcciones\...`, llegan ahí sin esfuerzo.

Arreglado en `a128171`: el archivo pasa a llamarse `7 - Modelo Enterprise Architect.eap` y la ruta baja a 98 caracteres, en línea con el resto. Qué contiene el modelo lo explica el LEEME, que es su lugar.

2 · Siete pruebas fallaban en un clon limpio

```
System.InvalidOperationException : Falta la clave de firma del token.
Configurá Jwt:Clave o PREDICTIT_JWT_CLAVE.
```

Las siete son las que levantan la aplicación real con `WebApplicationFactory`, así que arrancan el `Program` de verdad, y ese **se niega a arrancar sin clave de firma**. Hace bien: un valor por defecto en producción sería una clave conocida. El problema es de dónde salía la clave hasta ahora —de `appsettings.Development.json`, que está fuera del control de versiones— y que nada lo decía: en la máquina de trabajo las 257 pasaban y en un clon limpio no, sin que hubiera nada roto.

Es exactamente la clase de cosa que esta corrida busca: algo que funciona porque el que lo prueba tiene un archivo que el que clona no.

Arreglado con `backend/PredictIT.Tests/Infraestructura/ClaveDeFirmaDePrueba.cs`: un inicializador de módulo que pone una clave de usar y tirar **sólo si el entorno no trae la suya**. La integración continua sigue inyectando la propia y quien tenga una configurada la conserva. El resguardo de `Program` queda intacto, que es lo que importa; lo que cambia es que correr la suite deja de pedir un secreto.

3 · Dos copias del proyecto no conviven

Los contenedores llevan nombre fijo en `docker-compose.yml` —`predictit-sql`, `predictit-api`, `predictit-web`, `predictit-migraciones`— así que levantar el clon con el checkout de trabajo todavía creado da:

```
Conflict. The container name "/predictit-sql" is already in use
```

No se arregla, y es la decisión correcta. El nombre fijo es lo que hace que las instrucciones del manual y del guion de la defensa se puedan escribir —`docker logs predictit-api`, el `sqlcmd` del paso 8 de la demostración— sin que dependan del nombre de la carpeta desde la que se levantó. Quien clone el proyecto va a tener una sola copia; el que tiene dos es el autor, y le alcanza con `docker compose down` en una antes de levantar la otra.

Queda anotado acá porque es lo primero que aparece si alguien intenta repetir esta corrida sin bajar el stack de trabajo.

Lo que hay que traer a mano

Nada del código, y eso es el punto. Lo único que el repositorio no trae, a propósito, es el archivo `.env`: se copia de `.env.ejemplo`, que documenta cada variable y cómo generar las dos claves. Sin él, `docker compose` se detiene con un mensaje que dice qué falta y de dónde sacarlo, en lugar de arrancar con un valor por defecto.

Para correr `dotnet test` hace falta además **`PREDICTIT_DB_PASSWORD` en el entorno**, con la misma contraseña que quedó en el `.env`: las pruebas de integración se conectan a la base real como `predictit_app`, y esa contraseña no puede tener un valor por defecto porque es la de una base que cada uno crea. Está documentado en el manual de instalación.

Lo que esta corrida no prueba

- **Que la máquina de la defensa levante el sistema.** Se probó en la máquina de desarrollo, con las imágenes de Docker ya en caché. Una máquina sin ellas se las baja: son unos 2,3 GB de SQL Server más lo que se construye.

- **El respaldo y la restauración de la base de demostración.** Es el criterio 2 de H-53 y es del 30/10.
- **La etiqueta de versión.** Criterio 1, también del 30/10.
- **El recorrido de la demostración a mano.** Eso es H-52, y se hace con el reloj.

Fuentes

Beck, K. (2003). *Test-driven development: By example*. Addison-Wesley.

ISO/IEC. (2011). *Systems and software engineering. Systems and software Quality Requirements and Evaluation (SQuaRE). System and software quality models* (ISO/IEC 25010:2011).

National Institute of Standards and Technology. (2017). *Digital identity guidelines: Authentication and lifecycle management* (NIST Special Publication 800-63B). https://pages.nist.gov/800-63-3/sp800-63b.html

OWASP Foundation. (2021). *OWASP top 10: 2021*. https://owasp.org/Top10/